

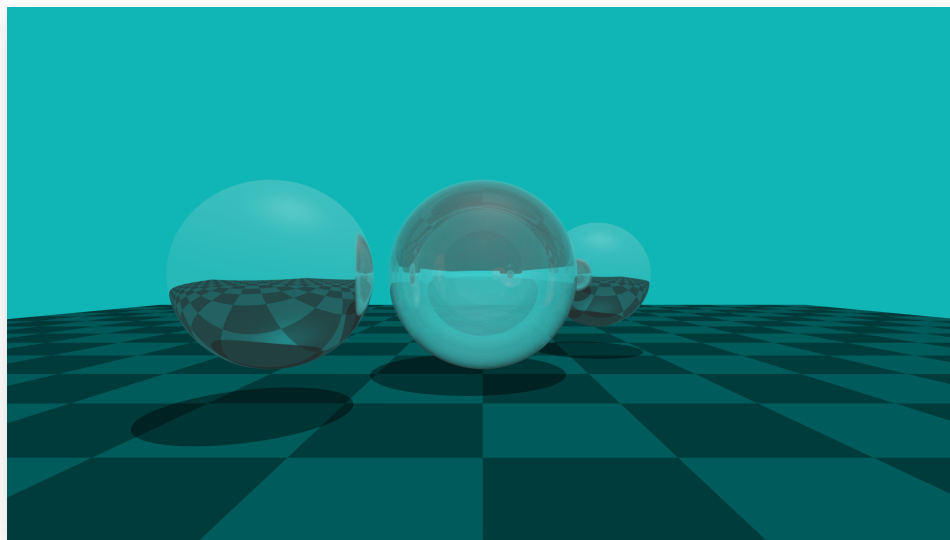
Sep 25, 2022

An Introduction to Ray Tracing In C

To practice my C programming skills and to learn more about computer graphics I wrote a simple raytracer (you can find the sourcecode on [github](#). My approach is based on [Raytracing in one Weekend](#), [scratchapixel.com](#) and [smallpt](#). All of these authors use C++ to implement the raytracing algorithm, but I decided to use C (C99 in particular) for the aforementioned reason.

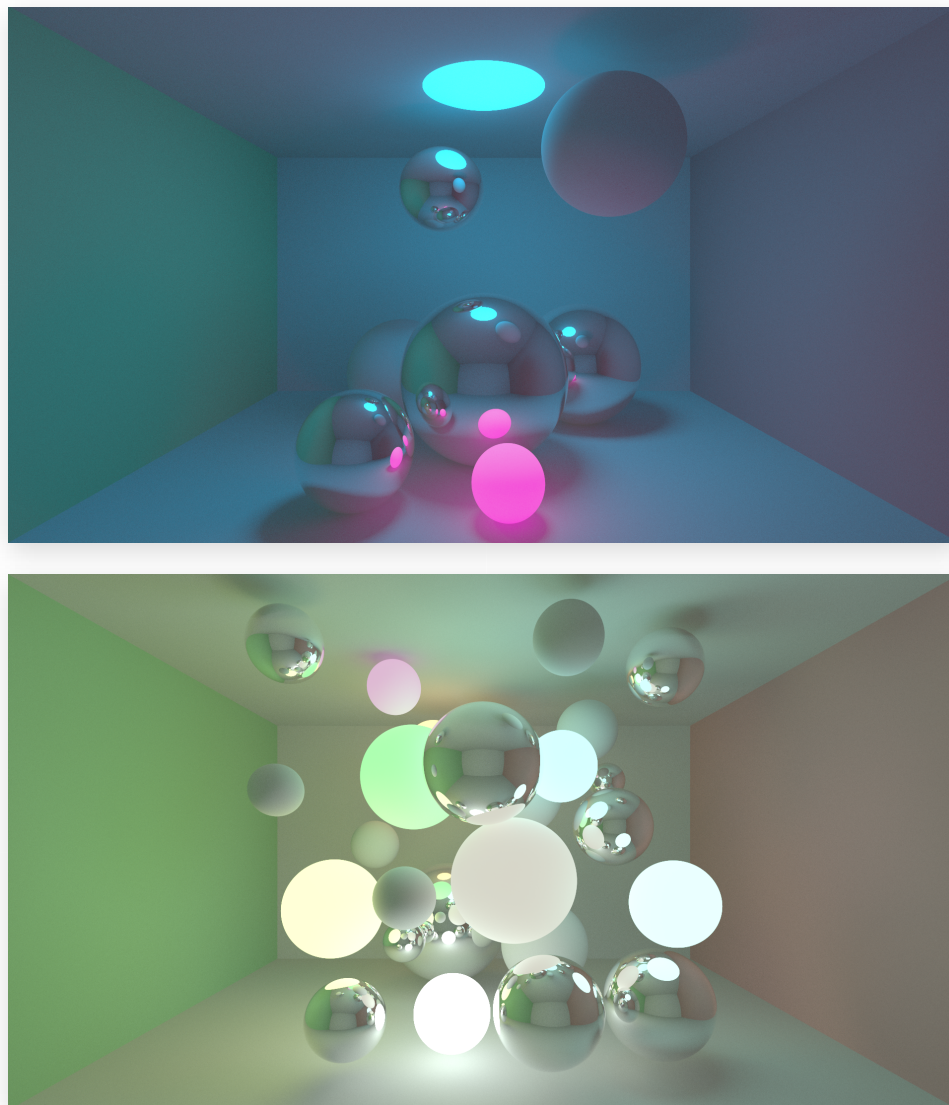
There was actually another motivation behind my effort to write a raytracer: I wanted to generate cool wallpapers to use on the computers that I work with every day. In the past I used the Windows 95 default wallpaper, which is just the color `#008080`, so I wanted to have a similar color for my new background. I also like the [bisexual lighting](#) color palette for it's cyberpunk feel, so that is something I also wanted to use

Here are is the rendering result I got using the algorithm from [wikipedia](#):



I also implemented [path tracing](#), which can produce more realistic results at the expense of being more computationally expensive. With Path Tracing you do not calculate the lighting with a formula, instead you integrate over all the illuminance that arrives at a single point on a surface. This method naturally simulates phenomenon like global illumination, caustics and soft shadows.

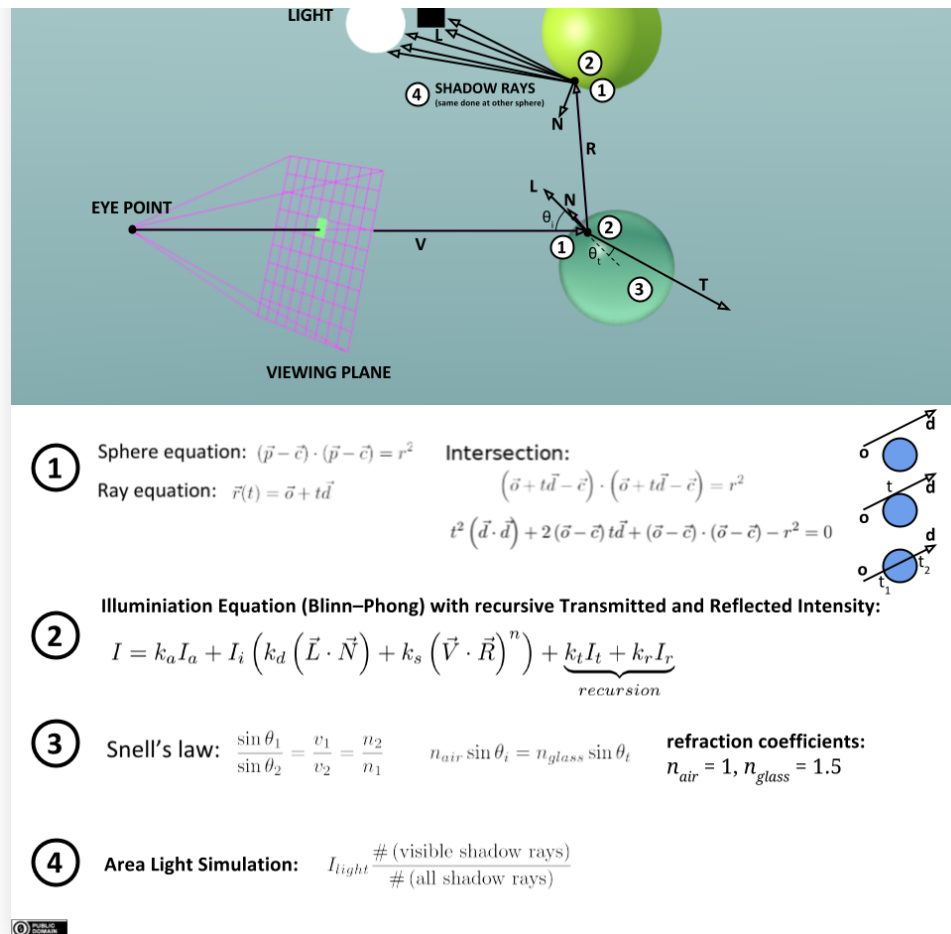
To produce high-quality, noise free images a very high number of rays need to be simulated, leading to very long rendering times. Raytracing is **embarrassingly parallel**, significant speedup could therefore be achieved by moving the computation to the GPU, for example by using OpenCL or a OpenGL compute shader. Right now my implementation only uses OpenMP for parallelisation so there is still room for improvement.



This image was rendered using 8192 samples per pixel.

Raytracing works by tracing a ray from a simulated eye through a pixel into a scene where the ray is then either absorbed, reflected and refracted by the objects in the scene, which then determines the color of the pixel. The algorithm is probably best explained by this graphic, which I found on [wikipedia](https://en.wikipedia.org/wiki/Ray_tracing):





Impressum & Datenschutzerklärung

[Edit this page on GitHub](#)

Site last updated on Sep 13, 2026 at 07:22 UTC

© 2026 Jakob Maier